

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 2 Practical No: 9

Roll no: 49 PRN No: 12414235 Name of Student: Saloni Tiwari

Subject Name: Deep Learning

Problem Statement:

Perform Image classification using Pretrained Vision Transformer model and compare its performance with CNN Model.

Algorithm:

- **Import Libraries:** Import TensorFlow, Transformers, Scikit-learn, NumPy, Matplotlib and other required libraries.
- **Load Dataset:** Load the CIFAR-10 dataset, containing 50,000 training images and 10,000 testing images.
- **Preprocess Data:** Normalize image pixel values to the range [0, 1] and define the ten image classes.
- **Visualize Images:** Display sample images from the CIFAR-10 dataset with their class labels.
- **Build CNN Model:** Create a Convolutional Neural Network using convolution, max-pooling, flattening and dense layers.
- **Train CNN:** Train the CNN on the full training dataset for five epochs, using 10% of the training data for validation.
- **Evaluate CNN:** Predict the classes of test images and calculate accuracy and classification metrics.
- **Load Pre-trained ViT:** Load the pre-trained Vision Transformer model and its image processor, replacing the original classification head with a ten-class output layer.
- **Prepare ViT Data:** Select 5,000 training images and 1,000 testing images. Resize and preprocess images in small batches to reduce memory usage.
- **Fine-tune ViT:** Compile and fine-tune the pre-trained ViT model on the selected training subset for two epochs.
- **Evaluate and Compare:** Evaluate ViT on the selected test subset and compare CNN and ViT using accuracy, precision, recall and F1-score. Visualize the accuracy comparison.

Code:

```
# Block 1: Install required libraries
!pip -q install tf-keras transformers scikit-learn

# Block 2: Import libraries
import tensorflow as tf
import tf_keras
from tensorflow.keras import layers, models
from transformers import ViTImageProcessor, TFViTForImageClassification
from sklearn.metrics import accuracy_score, classification_report
from sklearn.metrics import precision_score, recall_score, f1_score
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import math

# Block 3: Load and preprocess CIFAR-10 dataset
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.cifar10.load_data()
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
class_names = [
    "airplane", "automobile", "bird", "cat", "deer",
    "dog", "frog", "horse", "ship", "truck"
]
print("Training data:", x_train.shape)
print("Testing data:", x_test.shape)

# Block 4: Visualize sample images
plt.figure(figsize=(10, 4))
for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(x_train[i])
    plt.title(class_names[int(y_train[i][0])])
    plt.axis("off")
plt.tight_layout()
plt.show()

# Block 5: Build the CNN model
cnn = models.Sequential([
    layers.Conv2D(32, 3, activation="relu", input_shape=(32, 32, 3)),
    layers.MaxPooling2D(),
    layers.Conv2D(64, 3, activation="relu"),
    layers.MaxPooling2D(),
    layers.Flatten(),
    layers.Dense(128, activation="relu"),
    layers.Dense(10, activation="softmax")
])
cnn.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
cnn.summary()

# Block 6: Train the CNN model
cnn_history = cnn.fit(
    x_train,
    y_train,
    epochs=5,
    batch_size=64,
    validation_split=0.1
)

# Block 7: Evaluate the CNN model
cnn_loss, cnn_accuracy = cnn.evaluate(x_test, y_test, verbose=0)
cnn_pred = np.argmax(cnn.predict(x_test, batch_size=64), axis=1)
print("CNN Accuracy:", accuracy_score(y_test.flatten(), cnn_pred))
print(classification_report(
```

```

y_test.flatten(),
cnn_pred,
target_names=class_names
))

# Block 8: Load the pre-trained Vision Transformer
processor = ViTImageProcessor.from_pretrained(
    "google/vit-base-patch16-224"
)
vit = TFViTForImageClassification.from_pretrained(
    "google/vit-base-patch16-224",
    num_labels=10,
    ignore_mismatched_sizes=True
)

# Block 9: Prepare ViT images in small batches to reduce RAM usage
batch_size = 4
def batch_generator(images, labels, batch_size=4, shuffle=False, repeat=False):
    while True:
        indices = np.arange(len(images))
        if shuffle:
            np.random.shuffle(indices)
        for start in range(0, len(images), batch_size):
            idx = indices[start:start + batch_size]
            batch_images = (images[idx] * 255).astype(np.uint8)
            pixel_values = processor(
                images=list(batch_images),
                return_tensors="np"
            )["pixel_values"]
            yield (
                pixel_values.astype(np.float32),
                labels[idx].astype(np.int32).reshape(-1)
            )
        if not repeat:
            break
train_ds = tf.data.Dataset.from_generator(
    lambda: batch_generator(
        x_train[:5000],
        y_train[:5000],
        batch_size=batch_size,
        shuffle=True,
        repeat=True
    ),
    output_signature=(
        tf.TensorSpec(shape=(None, 3, 224, 224), dtype=tf.float32),
        tf.TensorSpec(shape=(None,), dtype=tf.int32)
    )
)
test_ds = tf.data.Dataset.from_generator(
    lambda: batch_generator(
        x_test[:1000],
        y_test[:1000],
        batch_size=batch_size,
        shuffle=False
    ),
    output_signature=(
        tf.TensorSpec(shape=(None, 3, 224, 224), dtype=tf.float32),
        tf.TensorSpec(shape=(None,), dtype=tf.int32)
    )
)
print("ViT training images:", 5000)
print("ViT testing images:", 1000)

# Block 10: Fine-tune and evaluate the ViT model
vit.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=5e-5),
    loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
    metrics=["accuracy"]
)

```

```

vit_history = vit.fit(
    train_ds,
    epochs=2,
    steps_per_epoch=math.ceil(5000 / batch_size)
)

# Generate ViT predictions in batches
all_logits = []

for pixel_values, labels in test_ds:
    outputs = vit(pixel_values=pixel_values, training=False)
    all_logits.extend(outputs.logits.numpy())

vit_pred = np.argmax(np.array(all_logits), axis=1)
test_lab = y_test[:1000].flatten()

print("ViT Accuracy:", accuracy_score(test_lab, vit_pred))
print(classification_report(
    test_lab,
    vit_pred,
    target_names=class_names
))

# Block 11: Compare CNN and ViT performance on the same 1,000 test images
cnn_pred_subset = cnn_pred[:1000]
cnn_true = y_test[:1000].flatten()
vit_true = test_lab

results = pd.DataFrame({
    "Model": ["CNN", "ViT"],
    "Accuracy": [
        accuracy_score(cnn_true, cnn_pred_subset),
        accuracy_score(vit_true, vit_pred)
    ],
    "Precision": [
        precision_score(cnn_true, cnn_pred_subset, average="weighted", zero_division=0),
        precision_score(vit_true, vit_pred, average="weighted", zero_division=0)
    ],
    "Recall": [
        recall_score(cnn_true, cnn_pred_subset, average="weighted", zero_division=0),
        recall_score(vit_true, vit_pred, average="weighted", zero_division=0)
    ],
    "F1-Score": [
        f1_score(cnn_true, cnn_pred_subset, average="weighted", zero_division=0),
        f1_score(vit_true, vit_pred, average="weighted", zero_division=0)
    ]
})

print("\nCNN vs ViT Performance:")
print(results.round(4))

# Plot the accuracy comparison
plt.figure(figsize=(6, 4))
plt.bar(results["Model"], results["Accuracy"])
plt.title("CNN vs ViT Accuracy")
plt.xlabel("Model")
plt.ylabel("Accuracy")
plt.ylim(0, 1)
plt.tight_layout()
plt.show()

```

Output:

1] Loading and normalizing the CIFAR-10 training and testing datasets.

```
# Load the complete CIFAR-10 dataset and normalize pixel values
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.cifar10.load_data()

x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer',
               'dog', 'frog', 'horse', 'ship', 'truck']

print("Training data:", x_train.shape)
print("Testing data:", x_test.shape)

Training data: (50000, 32, 32, 3)
Testing data: (10000, 32, 32, 3)
```

2] Displaying sample CIFAR-10 images with their corresponding class labels.

```
# Display sample images from the training dataset
plt.figure(figsize=(10, 4))

for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(x_train[i])
    plt.title(class_names[int(y_train[i][0])])
    plt.axis("off")

plt.tight_layout()
plt.show()
```



3] Defining the CNN model and displaying its layer structure and parameter summary

```
# Create a CNN model for CIFAR-10 image classification
cnn = models.Sequential([
    layers.Conv2D(32, 3, activation='relu', input_shape=(32, 32, 3)),
    layers.MaxPooling2D(),
    layers.Conv2D(64, 3, activation='relu'),
    layers.MaxPooling2D(),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dense(10, activation='softmax')
])

cnn.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

cnn.summary()
```

*** /usr/local/lib/python3.13/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 30, 30, 32)	896
max_pooling2d (MaxPooling2D)	(None, 15, 15, 32)	0
conv2d_1 (Conv2D)	(None, 13, 13, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 6, 6, 64)	0
flatten (Flatten)	(None, 2304)	0
dense (Dense)	(None, 128)	295,040
dense_1 (Dense)	(None, 10)	1,290

Total params: 315,722 (1.20 MB)
Trainable params: 315,722 (1.20 MB)
Non-trainable params: 0 (0.00 B)

4] Training the CNN for five epochs and monitoring training and validation accuracy.

```
# Train the CNN on the full training dataset for five epochs
cnn_history = cnn.fit(
    x_train, y_train,
    epochs=5,
    batch_size=64,
    validation_split=0.1
)

Epoch 1/5
704/704 ————— 20s 14ms/step - accuracy: 0.4604 - loss: 1.4995 - val_accuracy: 0.5698 - val_loss: 1.2103
Epoch 2/5
704/704 ————— 3s 4ms/step - accuracy: 0.5987 - loss: 1.1467 - val_accuracy: 0.6078 - val_loss: 1.1193
Epoch 3/5
704/704 ————— 3s 5ms/step - accuracy: 0.6472 - loss: 1.0106 - val_accuracy: 0.6518 - val_loss: 0.9919
Epoch 4/5
704/704 ————— 5s 4ms/step - accuracy: 0.6796 - loss: 0.9204 - val_accuracy: 0.6806 - val_loss: 0.9323
Epoch 5/5
704/704 ————— 3s 4ms/step - accuracy: 0.7033 - loss: 0.8498 - val_accuracy: 0.6810 - val_loss: 0.9307
```

5] Evaluating CNN performance on the test dataset using accuracy and classification metrics.

```
# Evaluate the CNN on all test images and generate predictions
cnn_loss, cnn_accuracy = cnn.evaluate(x_test, y_test, verbose=0)
cnn_pred = np.argmax(cnn.predict(x_test), axis=1)

print("CNN Accuracy:", accuracy_score(y_test.flatten(), cnn_pred))
print(classification_report(y_test.flatten(), cnn_pred, target_names=class_names))
```

```
313/313 ————— 2s 4ms/step
CNN Accuracy: 0.6722
```

	precision	recall	f1-score	support
airplane	0.77	0.66	0.71	1000
automobile	0.70	0.88	0.78	1000
bird	0.54	0.55	0.55	1000
cat	0.61	0.37	0.46	1000
deer	0.64	0.59	0.61	1000
dog	0.52	0.67	0.59	1000
frog	0.68	0.81	0.74	1000
horse	0.72	0.77	0.74	1000
ship	0.87	0.67	0.76	1000
truck	0.73	0.76	0.74	1000
accuracy			0.67	10000
macro avg	0.68	0.67	0.67	10000
weighted avg	0.68	0.67	0.67	10000

6] Fine-tuning the pre-trained Vision Transformer and evaluating its classification performance.

```
• Epoch 1/2
1250/1250 [=====] - 311s 212ms/step - loss: 0.2750 - accuracy: 0.9286
Epoch 2/2
WARNING:tensorflow:Your input ran out of data; interrupting training. Make sure that your dataset or generator can generate enough samples for the data you are training on.
1250/1250 [=====] - 1s 737us/step - loss: 0.0000e+00 - accuracy: 0.0000e+00
ViT Accuracy: 0.955
```

	precision	recall	f1-score	support
airplane	0.99	0.99	0.99	103
automobile	0.98	0.97	0.97	89
bird	1.00	0.98	0.99	100
cat	0.95	0.81	0.87	103
deer	1.00	0.90	0.95	90
dog	0.80	0.94	0.87	86
frog	0.98	0.99	0.99	112
horse	0.90	1.00	0.95	102
ship	0.99	0.99	0.99	106
truck	0.96	0.97	0.97	109
accuracy			0.95	1000
macro avg	0.96	0.95	0.95	1000
weighted avg	0.96	0.95	0.96	1000

8] Comparing CNN and ViT using accuracy, precision, recall, and F1-score, with a visual accuracy comparison.

